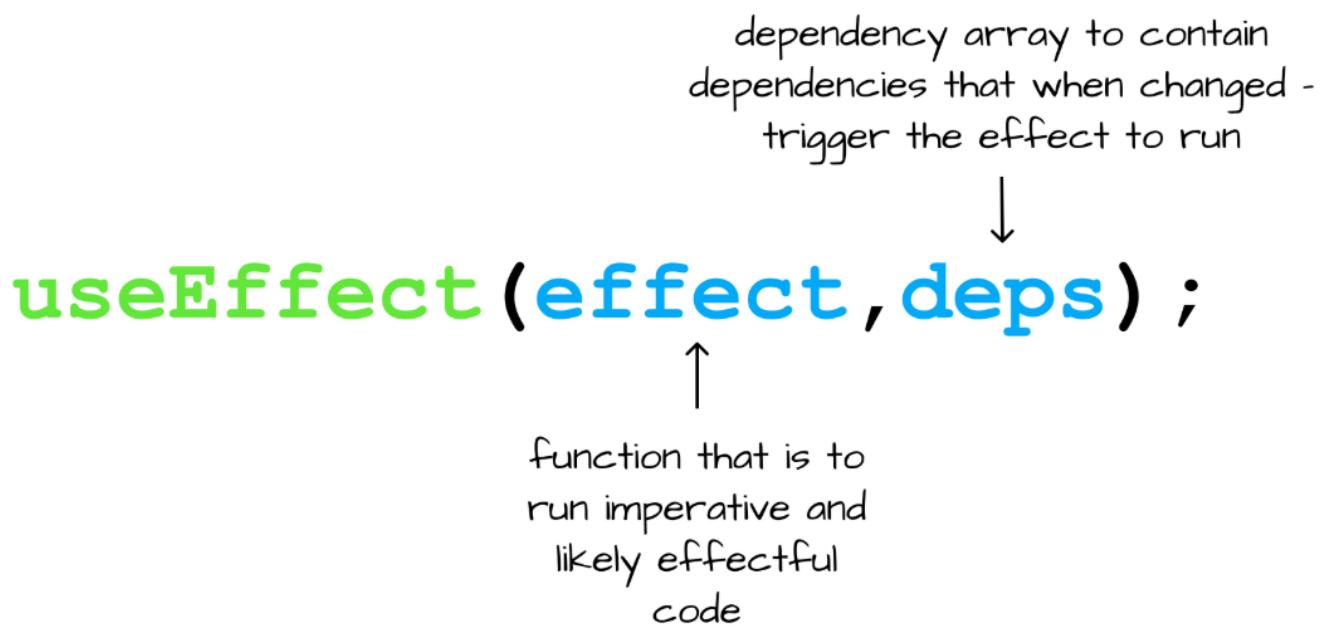


UseEffect

The useEffect Hook allows you to perform side effects in your components. A React Hook for handling side effects in functional components. Effects are executed after the browser paints the component instance on the screen, not immediately after render. That's why effects run async after the render has already painted. Side effects = anything that affects things outside the component:

- Some examples of side effects are: API calls, fetching data, subscriptions, directly updating the DOM, and timers.
- useEffect accepts two arguments. The second argument is optional. `useEffect(function, dependency array)`



1. No dependency passed:

```
useEffect(() => {  
  //Runs on every render  
});
```

2. An empty array : Run Only on Mount (componentDidMount)

```
useEffect(() => {  
  //Runs only on the first render  
}, []);
```

3. Props or state values : Run on Mount + When Dependencies Change (componentDidUpdate)

```
useEffect(() => {  
  //Runs on the first render  
  //And any time any dependency value changes  
}, [prop, state]);
```

4.Cleanup Logic (componentWillUnmount):

```
useEffect(() => {  
  const timer = setInterval(() => {  
    console.log('Running...');  
  }, 1000);  
  
  return () => {  
    clearInterval(timer);  
    console.log('Cleanup: component unmounted or dependencies changed');  
  };  
}, []);
```

5.Fetch API Data + Cleanup:

Effect Cleanup

Some effects require cleanup to reduce memory leaks.

Timeouts, subscriptions, event listeners, and other effects that are no longer needed should be disposed.

We do this by including a return function at the end of the useEffect Hook.

```
useEffect(() => {  
  const controller = new AbortController();  
  
  fetch('/data.json', { signal: controller.signal })  
    .then(res => res.json())  
    .then(data => console.log(data))  
    .catch(err => {  
      if (err.name === 'AbortError') {  
        console.log('Fetch aborted');  
      }  
    });  
  
  return () => {  
    controller.abort(); // Cancel fetch on unmount  
  };  
}, []);
```

6.Multiple Effects:

You can use `useEffect` more than once in a component:

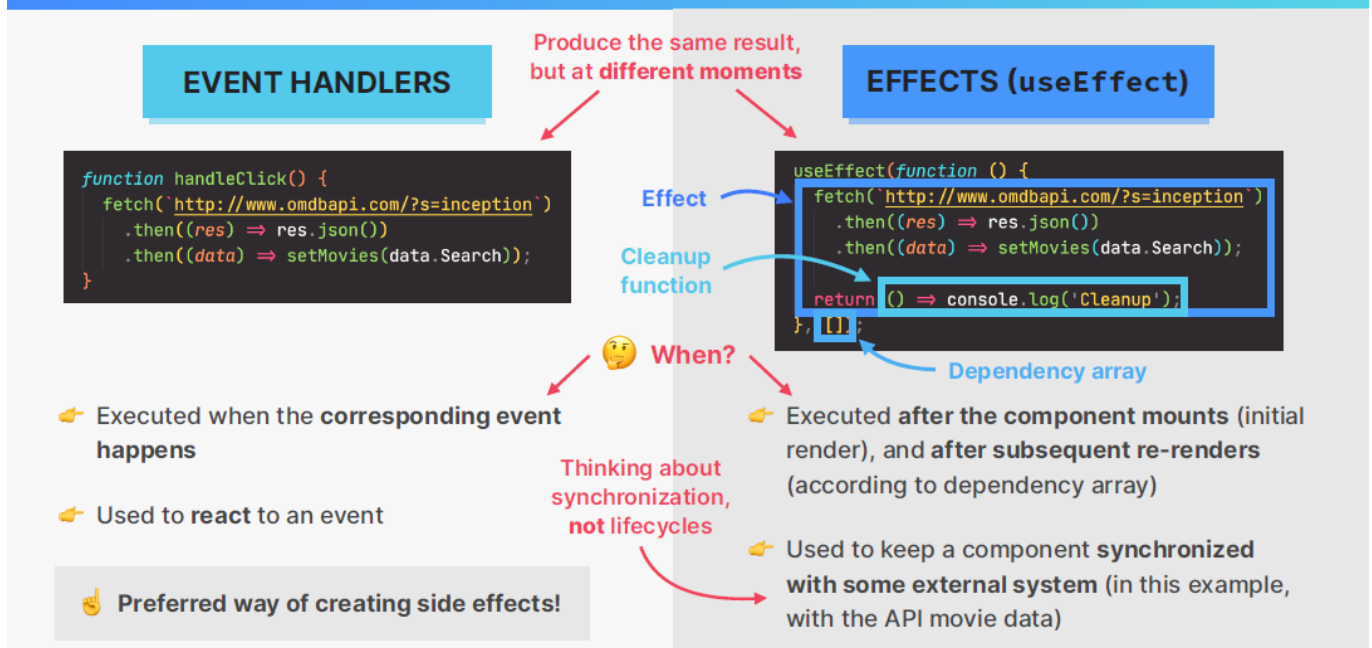
```
useEffect(() => {
  console.log('Fetch users');
}, []);

useEffect(() => {
  console.log('Log whenever count changes');
}, [count]);
```

Summary Table

Pattern	Equivalent to...	Syntax Example
On every render	<code>componentDidUpdate</code>	<code>useEffect(() => {})</code>
Only on mount	<code>componentDidMount</code>	<code>useEffect(() => {}, [])</code>
On unmount	<code>componentWillUnmount</code>	<code>return () => {...}</code> inside <code>useEffect</code>
On dependency change	<code>componentDidUpdate</code>	<code>useEffect(() => {}, [var])</code>

EVENT HANDLERS VS. EFFECTS



What is the useEffect Dependency Array?

The dependency array is the second argument you pass to `useEffect`:

```
useEffect(() => {
  // effect code here
}, [/* dependencies */]);
```

It tells React when to run the effect by specifying variables that the effect depends on.

THE DEPENDENCY ARRAY

- 👉 By default, effects run **after every render**. We can prevent that by passing a **dependency array**
- 👉 Without the dependency array, React doesn't know **when** to run the effect
- 👉 *Each time one of the dependencies changes, the effect will be executed again*
- 👉 Every **state variable** and **prop** used inside the effect **MUST** be included in the dependency array

```
const title = props.movie.Title;
const [userRating, setUserRating] = useState('');

useEffect(
  function () {
    if (!title) return;
    document.title = `${title} ${
      userRating && `(Rated ${userRating} ★)`
    }`;
    return () => (document.title = 'usePopcorn');
  },
  [title, userRating]
);
```

Otherwise, we get a **"stale closure"**. We will go more into depth in a future section 👉

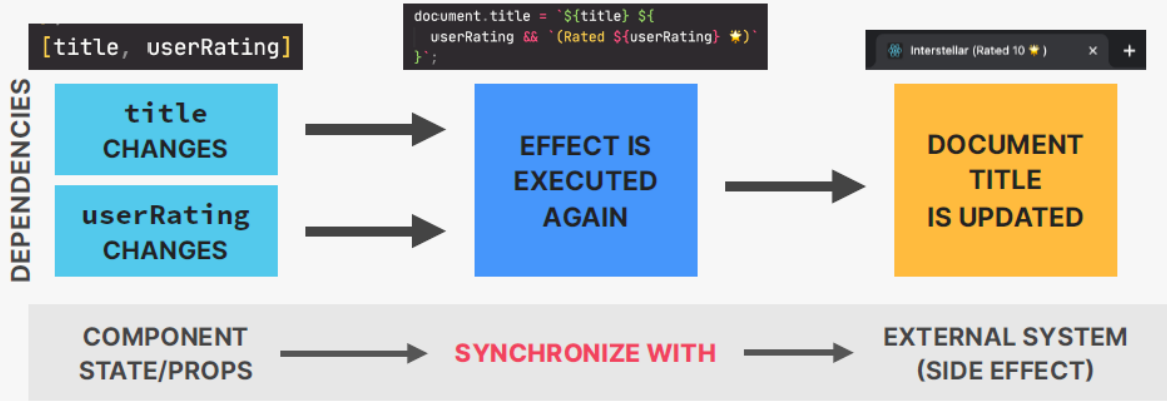
How It Works

1. **Empty array []:**
Runs the effect only **once** after the first render (similar to `componentDidMount`).
2. **No array (no second argument):**
Runs the effect **after every render** — on every update and mount.
3. **Array with dependencies [dep1, dep2]:**
Runs the effect **only when any dependency changes** since the last render.

USEEFFECT IS A **SYNCHRONIZATION** MECHANISM

THE MECHANICS OF EFFECTS

- 👉 `useEffect` is like an **event listener** that is listening for one dependency to change. **Whenever a dependency changes, it will execute the effect again**
- 👉 Effects **react** to updates to state and props used inside the effect (the dependencies). So **effects** are “**reactive**” (like state updates re-rendering the UI)



SYNCHRONIZATION AND LIFECYCLE



👉 We can use the dependency array to run effects **when the component renders or re-renders**



SYNCHRONIZATION



LIFECYCLE

```
useEffect(fn, [x, y, z]);
```

```
useEffect(fn, []);
```

```
useEffect(fn);
```



Effect synchronizes with **x, y, and z**

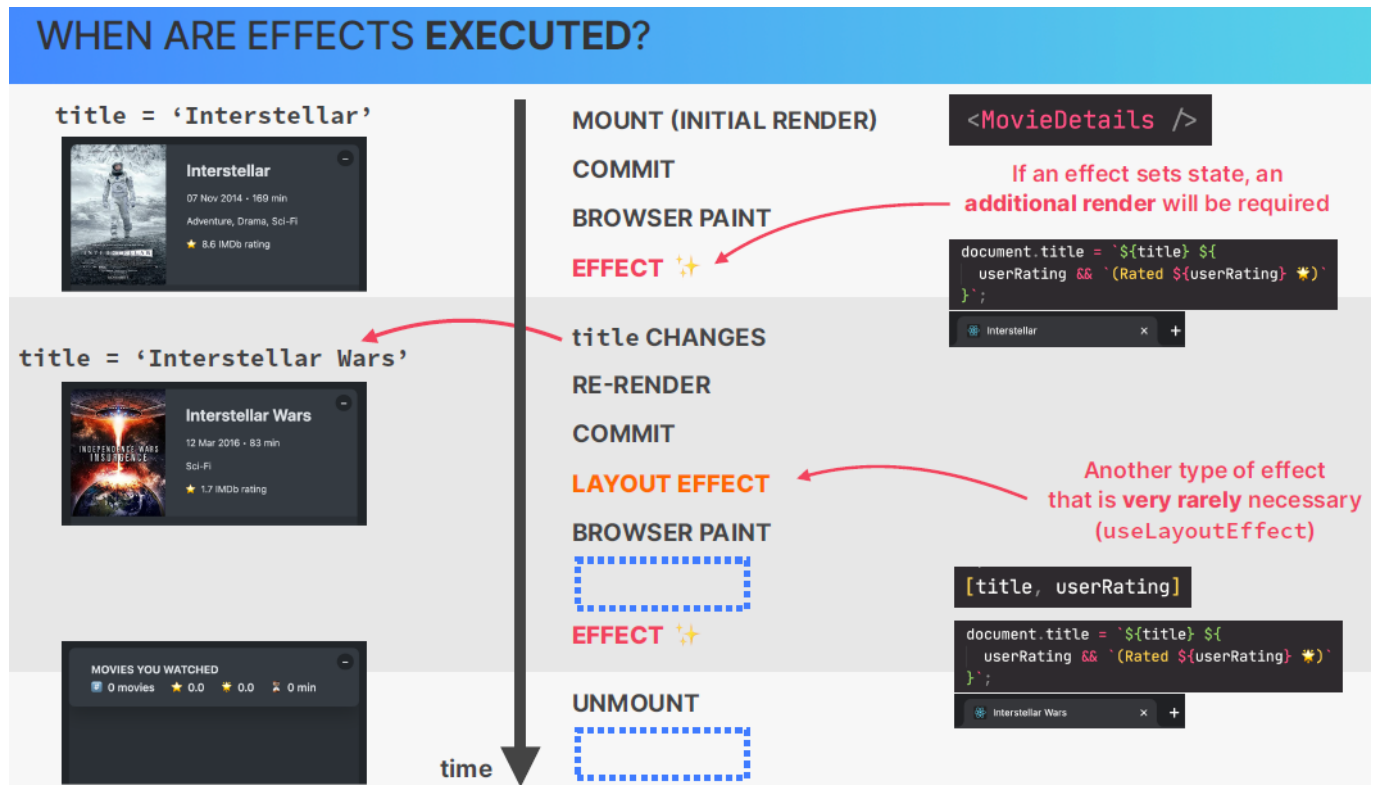
Effect synchronizes with **no state/props**

Effect synchronizes with **everything**

Runs on **mount** and **re-renders** triggered by updating **x, y, or z**

Runs only on **mount** (initial render)

Runs on **every render** (usually bad 🚫)



useLayoutEffect : Layout effect

useLayoutEffect: Like `useEffect`, but it runs synchronously after DOM updates but before the browser paints. The React JS `useLayoutEffect` works similarly to `useEffect` but rather works asynchronously like the `useEffect` hook, it fires synchronously after all DOM loading is done loading. This is useful for synchronously re-rendering the DOM and also to read the layout from the DOM. But to prevent blocking the page loading, we should always use the `useEffect` hook.

Use it when you need to:

- Measure layout (e.g., element size)
- Make DOM changes that should happen before the user sees them
- Avoid flickering UI updates

🧠 Key Differences: `useEffect` vs `useLayoutEffect`

Feature	<code>useEffect</code>	<code>useLayoutEffect</code>
Timing	After paint	Before paint (after DOM changes)
Blocking UI?	No	Yes — it blocks painting
Use case	API calls, logging, subscriptions	DOM reads/writes, preventing layout shift
Performance impact	Minimal	Can hurt performance if overused

Example :-

Make DOM Changes Before User Sees Them

Use case: Modify styles immediately to prevent layout shifts. here Without `useLayoutEffect`, the component might briefly appear gray before turning gold.

```
import React, { useLayoutEffect, useRef } from 'react';

const InstantStyle = () => {
  const ref = useRef();

  useLayoutEffect(() => {
    if (ref.current) {
      ref.current.style.backgroundColor = 'gold'; // change before paint
      ref.current.style.padding = '20px';
    }
  }, []);

  return (
    <div
      ref={ref}
      style={{ transition: 'background 1s ease', backgroundColor: 'lightgray' }}
    >
      Background updated instantly (no flash)
    </div>
  );
};

export default InstantStyle;
```

useRef

- The `useRef` Hook allows you to persist values between renders.
- It can be used to store a mutable value that does not cause a re-render when updated.
- It can be used to access a DOM element directly.

✓ Example 1: DOM Access

```
jsx                                                                    Copy Edit

import React, { useRef, useEffect } from 'react';

const InputFocus = () => {
  const inputRef = useRef(); // create a ref

  useEffect(() => {
    inputRef.current.focus(); // focus the input after mount
  }, []);

  return <input ref={inputRef} placeholder="I am auto-focused!" />;
};
```

- ♦ `inputRef.current` gives direct access to the DOM input element.
- ♦ It's set when the element is rendered and is persistent across renders.

✓ Example 2: Store a Mutable Value

```
jsx                                                                    Copy Edit

import React, { useRef, useState, useEffect } from 'react';

const Timer = () => {
  const countRef = useRef(0);
  const [dummy, setDummy] = useState(0); // just to force re-renders

  useEffect(() => {
    const interval = setInterval(() => {
      countRef.current += 1; // value updates
      console.log('Count:', countRef.current);
    }, 1000);

    return () => clearInterval(interval);
  }, []);

  return (
    <div>
      <p>Check console for ref count (doesn't cause re-renders)</p>
      <button onClick={() => setDummy(dummy + 1)}>Re-render</button>
    </div>
  );
};
```

- ♦ `useRef` is used here to store mutable state without re-rendering the component.

For example :- If we tried to count how many times our application renders using the `useState` Hook, we would be caught in an infinite loop since this Hook itself causes a re-render.

To avoid this, we can use the `useRef` Hook.

```
import { useState, useEffect, useRef } from "react";
import ReactDOM from "react-dom/client";
```



```
function App() {
  const [inputValue, setInputValue] = useState("");
  const count = useRef(0);

  useEffect(() => {
    count.current = count.current + 1;
  });

  return (
    <>
      <input
        type="text"
        value={inputValue}
        onChange={(e) => setInputValue(e.target.value)}
      />
      <h1>Render Count: {count.current}</h1>
    </>
  );
}
```

 `useRef` + `useLayoutEffect` : Perfect Combo

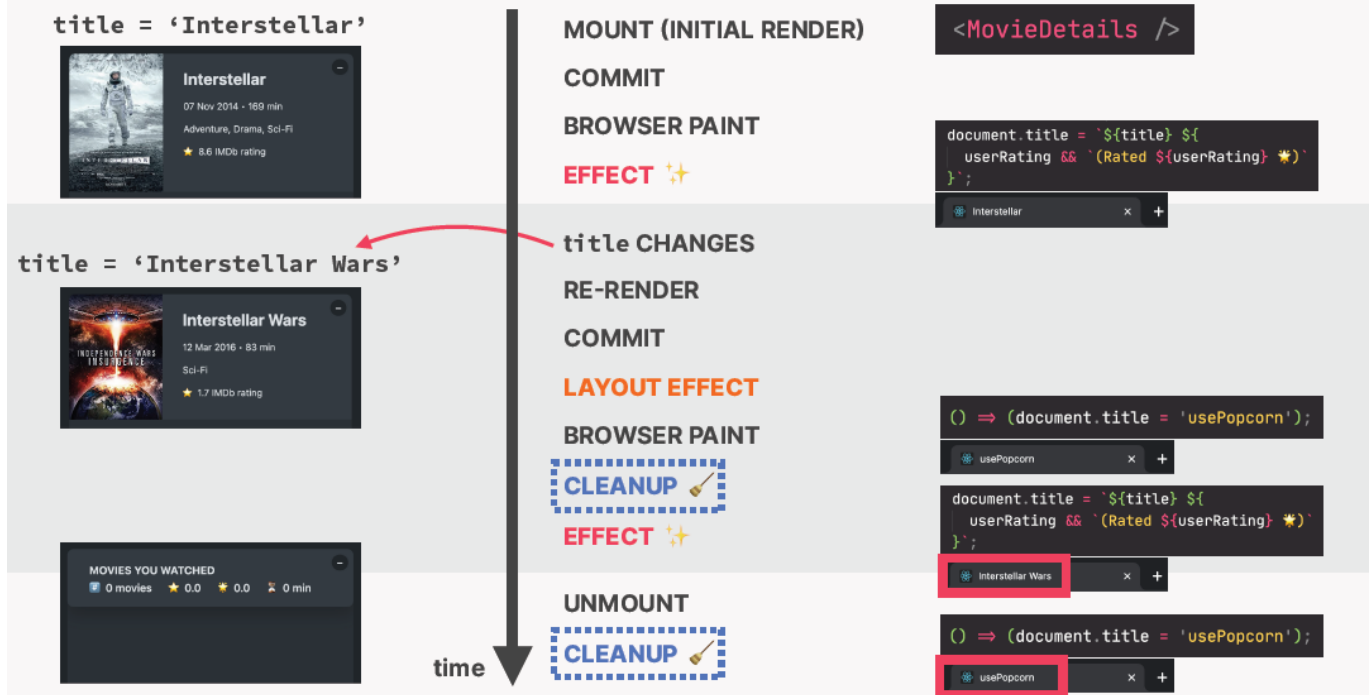
They often work **together** when:

- You want to **read or change the DOM** before the user sees it
- You need to **measure or reposition** elements after rendering

Summary

Hook	Purpose	Triggers Re-render?	DOM Access
<code>useRef()</code>	Store DOM reference or mutable value	 No	 Yes
<code>useLayoutEffect</code>	Run logic after DOM update, before paint	 No (by itself)	 Yes

WHEN ARE EFFECTS EXECUTED?



THE CLEANUP FUNCTION

USEEFFECT CLEANUP FUNCTION

👉 Function that we can return from an effect (optional)

👉 Runs on two different occasions:

- 1 Before the effect is executed again
- 2 After a component has unmounted

👉 Necessary whenever the side effect keeps happening after the component has been re-rendered or unmounted

👉 Each effect should do **only one thing!** Use **one useEffect** hook for each side effect. This makes effects easier to clean up

COMPONENT
RENDERS

Execute effect if
dependency array
includes updated data

COMPONENT
UNMOUNTS

Execute
cleanup function

Examples

✨ EFFECT

🧹 POTENTIAL
CLEANUP

- 👉 HTTP request → Cancel request
- 👉 API subscription → Cancel subscription
- 👉 Start timer → Stop timer
- 👉 Add event listener → Remove listener